VIT Vidyalankar Institute of Technology Accredited A+ by NAAC (Autonomous College Affiliated to University of Mumbai)	***Assignment-5 & 6***				
Title of the assignment	**AI Foundations through Competitive Coding**				
Semester: V	**Course:** Artificial Intelligence	**Scheme:** R-2023	**Code:** PEEC01T	**Module:** 3	**Max Marks:** 30
CO2	To know and use various problem-solving methods	**Date of Submission:** 27-08-26			
Expected time for assignment completion: 3 hrs		**Time taken for assignment completion:** 4 hrs			

Q.No	Questions	Bloom	Question Type	Difficulty	Industry Context	Expected Time	Resources	Assessment Evidence	Rubric Focus (Marks Breakdown)
1	Task 1 - Search Algorithms (Uninformed Search): Solve any two BFS/DFS-based problems on LeetCode. Analysis: How could an uninformed search algorithm help a food-delivery robot navigate through city streets? (approx. 100-150 words)	Applying	Coding + Analytical	Medium	Robotics navigation / food-delivery logistics	60 min	LeetCode BFS/DFS problem sets	Code screenshots/links + written analysis	Concept(2): correct algorithm use; Technical(2): working code; Application(2): navigation context; Analysis(2): depth of answer; Presentation(2): clarity
2	Task 2 - Logic & Constraints (N-Queens Problem): Implement and solve variations of the N-Queens problem on HackerRank. Analysis: In what way is solving N-Queens similar to how an AI agent solves Sudoku puzzles? (approx. 100-150 words)	Applying / Analysing	Coding + Analytical	Medium-Hard	Constraint satisfaction problems (e.g., puzzle solvers, scheduling)	60 min	HackerRank N-Queens problem set	Code screenshots/links + written analysis	Concept(2): correct constraint modelling; Technical(2): working code; Application(2): Sudoku analogy; Analysis(2): reasoning depth; Presentation(2): clarity
3	Task 3 - Optimization (Informed Search): Solve Minimum Cost Path or other informed-search problems on CodeChef. Analysis: How would you adapt such an optimization algorithm	Applying / Evaluating	Coding + Analytical	Hard	Drone navigation and obstacle avoidance	60 min	CodeChef informed-search / optimization problems	Code screenshots/links + written analysis	Concept(2): correct optimization approach; Technical(2): working code; Application(2): drone-navigation context; Analysis(2):

Q.No	Questions	Bloom	Question Type	Difficulty	Industry Context	Expected Time	Resources	Assessment Evidence	Rubric Focus (Marks Breakdown)
	for a drone to avoid obstacles during navigation? (approx. 100-150 words)								reasoning depth; Presentation(2): clarity

All three tasks carry equal marks of 10. Submit code solutions (screenshots and links) along with written answers (approx. 100-150 words each) to all analysis questions.

CO2	To know and use various problem-solving methods

I commit to submitting original work, free from plagiarism or copying from the internet or peers, and to honouring deadlines for a fair and transparent assessment

VIRAJ PRAMOD MAHALE
24108B0046
EXCS B

TASK 1 LEET CODE BFS AND DFS :

DSF Given an m x n 2D binary grid grid which represents a map of '1's (land) and '0's (water), return the number of islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1: Input: grid = [["1","1","1","1","0"], ["1","1","0","1","0"], ["1","1","0","0","0"], ["0","0","0","0","0"]]

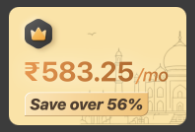
Output: 1

Description | Accepted | Editorial | Solutions | Submiss

All Submissions

Accepted 49 / 49 testcases passed

CodeWithSujall submitted at Oct 09, 2026 20:39



Unlock the Full LeetCode Experience

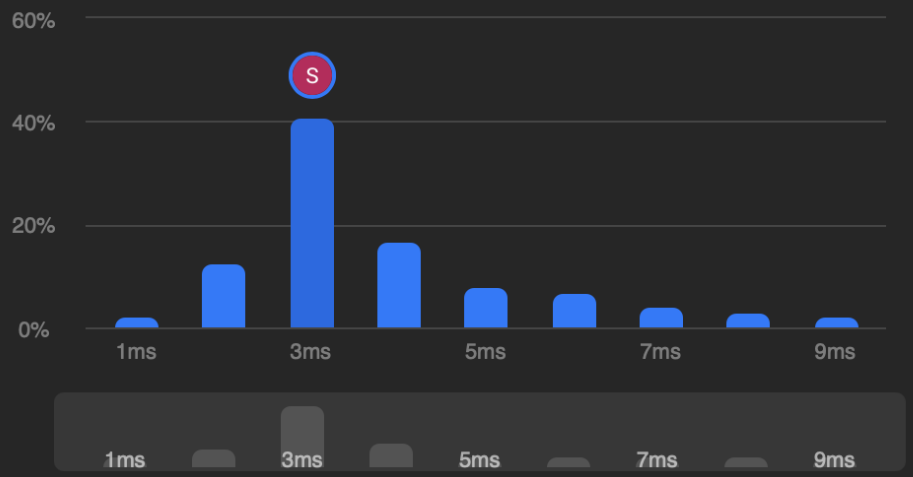
Company problems, Ask Leet, and expert editorials — al...

Runtime

3 ms | Beats **87.36%**

Memory

52.69 MB | Beats **13.93%**



Code | Java

Code

Java | Auto

```
1
2  class Solution {
3      public int numIslands(char[][] grid) {
4          int count = 0;
5
6          for (int i = 0; i < grid.length; i++) {
7              for (int j = 0; j < grid[0].length; j++) {
8                  if (grid[i][j] == '1') {
9                      count++;
10                     dfs(grid, i, j);
11                 }
12             }
13         }
14
```

Saved | Ln 1, Col 1

Testcase | Test Result

Case 1 | Case 2 +

grid =

```
[["1","1","1","1","0"],["1","1","0","1","0"],
["1","1","0","0","0"],["0","0","0","0","0"]]
```

[Number of Islands - LeeCode]

```
#include <stdio.h>

void dfs(char** grid, int r, int c, int rows, int cols) {
    // Boundary check or water check
```

```
    if (r < 0 || r >= rows || c < 0 || c >= cols || grid[r][c] == '0')
        return;

    // Mark current land as visited
    grid[r][c] = '0';

    // Visit all 4 directions
    dfs(grid, r + 1, c, rows, cols);
    dfs(grid, r - 1, c, rows, cols);
    dfs(grid, r, c + 1, rows, cols);
    dfs(grid, r, c - 1, rows, cols);
}

int numIslands(char** grid, int gridSize, int* gridColSize) {
    int count = 0;
    int rows = gridSize;
    int cols = gridColSize[0];

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {

            if (grid[i][j] == '1') {
                count++;

                // Mark the entire island as visited
```

```
                dfs(grid, i, j, rows, cols);
            }
        }
    }

    return count;
}
```

BSF

102. Binary Tree Level Order Traversal

Medium Topics

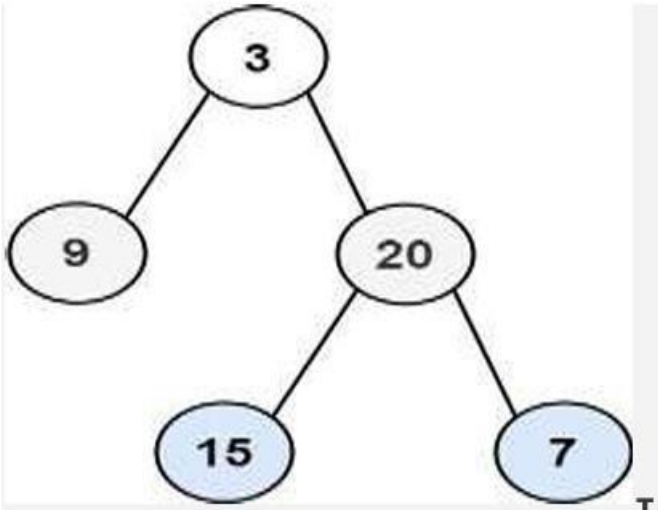


Companies

Hint

Given the root of a binary tree, return *the level order traversal of its nodes' values*. (i.e., from left to right, level by level).

Example 1:



Input root = [3,9,20,null,null,15,7]

Output [[3],[9,20],[15,7]]

Description | Accepted | Editorial | Solutions | Submiss

All Submissions

Accepted 29 / 29 testcases passed

CodeWithSujall submitted at Oct 09, 2026 16:15

₹583.25/mo
Save over 56%

Unlock the Full LeetCode Experience

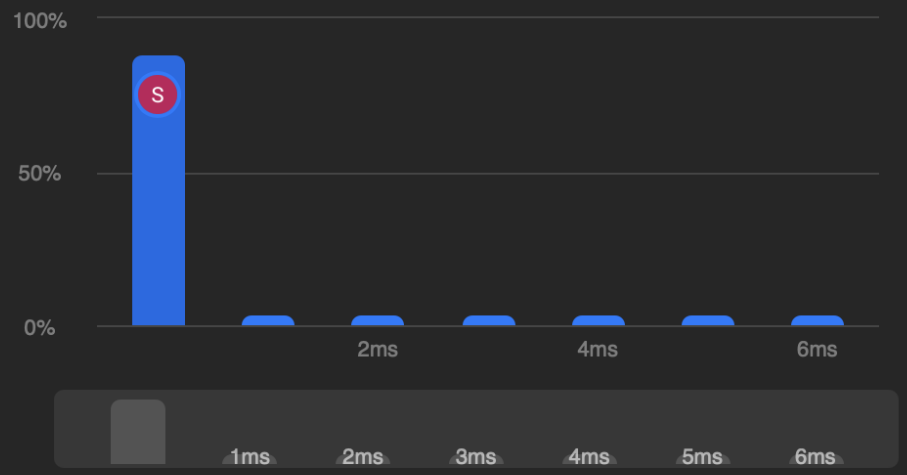
Company problems, Ask Leet, and expert editorials — al...

Runtime

0 ms | Beats **100.00%**

Memory

46.69 MB | Beats **72.62%**



Code | Java

Code

Java | Auto

```
8            slow = slow.next;
9            fast = fast.next.next;
10
11           if (slow == fast) {
12               return true;
13           }
14       }
15
16       return false;
17   }
18 }
19
```

Saved Ln 19, Col 1

Testcase | Test Result

Case 1 Case 2 Case 3

head =

```
[3,2,0,-4]
```

pos =

```
1
```

Binary Tree Level Order Traversal - LeetCode

```
/**
 * Definition for a binary tree node.
```

```
 * struct TreeNode {
 *     int val;
 *     struct TreeNode *left;
 *     struct TreeNode *right;
 * };
 */

/**
 * Return an array of arrays of size *returnSize.
 * The sizes of the arrays are returned in *returnColumnSizes.
 */

int** levelOrder(struct TreeNode* root, int* returnSize, int** returnColumnSizes) {
    if (root == NULL) {
        *returnSize = 0;
        *returnColumnSizes = NULL;
        return NULL;
    }

    int** result = malloc(2000 * sizeof(int*));
    *returnColumnSizes = malloc(2000 * sizeof(int));

    struct TreeNode* queue[2000];
    int front = 0, rear = 0;
```

```
queue[rear++] = root;

int level = 0;

while (front < rear) {
    int levelSize = rear - front;

    result[level] = malloc(levelSize * sizeof(int));
    (*returnColumnSizes)[level] = levelSize;

    for (int i = 0; i < levelSize; i++) {
        struct TreeNode* current = queue[front++];

        result[level][i] = current->val;

        if (current->left != NULL)
            queue[rear++] = current->left;

        if (current->right != NULL)
            queue[rear++] = current->right;
    }

    level++;
}
```

```
    *returnSize = level;

    return result;
}
```

Vijay Pramod Mahale
2410B30046
EX:5-B

DOMS | Page No.
Date / /

AI Assignment-5

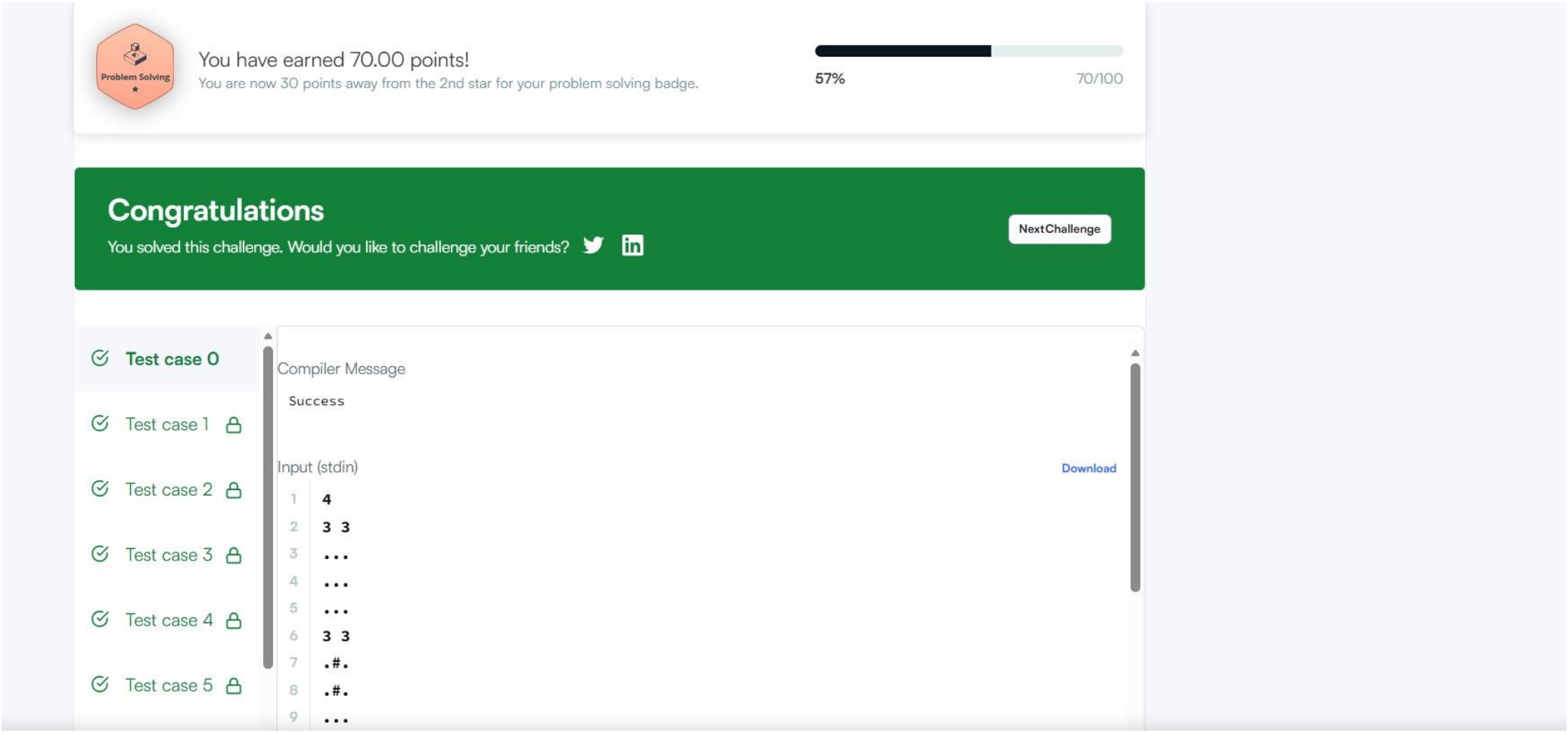
Task 1 :- Search algorithm (Uninformed Search : Method)

→ An Uniformed Search algorithm, or blind search, helps a food delivery robot navigate by treating the city and as a complex mathematical graph. Because the robot lacks a "Sense of direction" to guide it specifically toward the customer location, it explore the city by expanding outward systematically in every direction from its starting point. Using a method like Breadth first search, the robot map out every intersection one block away, then two blocks away, and so forth, until the destination. Alternatively, Uniform Cost Search allows the robot to calculate the actual travel distance for every street segment, prioritizing routes with the lowest physical cost. While these algorithm mathematically guarantee that the robot will eventually find the absolute shortest delivery route, they are computationally heavy. The robot must map countless irrelevant streets - including those in the complete opposite direction before finally arriving at the correct doorstep with the customer's order.

TASK 2

Queens on Board You have an N * M chessboard on which some squares are blocked out. In how many ways can you place one or more queens on the board, such that, no two queens attack each other? Two queens attack each other, if one can reach the other by moving horizontally, vertically, or diagonally without passing over any blocked square. At most one queen can be placed on a square. A queen cannot be placed on a blocked square. Input Format The first line contains the number of test cases T. T test cases follow. Each test case contains integers N and M on the first line. The following N lines contain M characters each, and represent a board. A '#' represents a blocked square and a '.' represents an unblocked square. Constraints 1 <= T <= 100 1 <= N <= 50 1 <= M <= 5 Output Format Output T lines containing the required answer for each test case.

As the answers can be really large, output them modulo 109+7. Sample Input 4 3 3 ... ... 3 3 .#. .#. ... 2 4 .#. ..... 1 1 # Sample Output 17 18 14 0



[Queens on Board | HackerRank]

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
```

```
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    static final int MOD = 1000000007;

    /*
     * Complete the 'queensBoard' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts STRING_ARRAY board as parameter.
     */

    public static int queensBoard(List<String> board) {
        int n = board.size();
        int m = board.get(0).length();

        Map<String, Long> dp = new HashMap<>();
        dp.put("0,0,0", 1L);
```

```
for (int r = 0; r < n; r++) {
    String row = board.get(r);

    // Find unblocked cells
    int openMask = 0;
     for (int c = 0; c < m; c++) {
         if (row.charAt(c) == '.') {
             openMask |= (1 << c);
         }
    }

    // Generate all possible queen placements in this row
    List<Integer> placements = new ArrayList<>();
    for (int q = 0; q < (1 << m); q++) {
        // Queen cannot be placed on #
        if ((q & ~openMask) != 0) {
            continue;
        }
        // Check horizontal attacks
        if (validRow(row, q, m)) {
            placements.add(q);
        }
    }

    Map<String, Long> next = new HashMap<>();
```

```
    for (Map.Entry<String, Long> entry : dp.entrySet()) {
        String[] parts = entry.getKey().split(",");
        int v = Integer.parseInt(parts[0]);
        int dl = Integer.parseInt(parts[1]);
        int dr = Integer.parseInt(parts[2]);
        long ways = entry.getValue();

        for (int q : placements) {
            // If a queen in the current row is attacked by a previous queen
            if ((q & v) != 0 || (q & dl) != 0 || (q & dr) != 0) {
                continue;
            }

            int continueMask = openMask & ~q;
            int newV = q | (v & continueMask);
            int newDl = ((q << 1) | ((dl & continueMask) << 1)) & ((1 << m) - 1);
            int newDr = (q >> 1) | ((dr & continueMask) >> 1);

            String key = newV + "," + newDl + "," + newDr;
            next.put(key, (next.getOrDefault(key, 0L) + ways) % MOD);
        }
    }
    dp = next;
}
```

```
    long answer = 0;
    for (long ways : dp.values()) {
        answer = (answer + ways) % MOD;
    }

    // Remove the empty arrangement (no queens)
    answer = (answer - 1 + MOD) % MOD;

    return (int) answer;
}

private static boolean validRow(String row, int q, int m) {
    for (int c = 0; c < m; c++) {
        if ((q & (1 << c)) == 0) {
            continue;
        }
        // Look to the left
        for (int x = c - 1; x >= 0 && row.charAt(x) == '.'; x--) {
            if ((q & (1 << x)) != 0) {
                return false;
            }
        }
        // Look to the right
        for (int x = c + 1; x < m && row.charAt(x) == '.'; x++) {
```

```
            if ((q & (1 << x)) != 0) {
                return false;
            }
        }
    }
    return true;
  }
}

public class Solution {
  public static void main(String[] args) throws IOException {
    BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
    BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

    int t = Integer.parseInt(bufferedReader.readLine().trim());

    IntStream.range(0, t).forEach(tItr -> {
      try {
        String[] firstMultipleInput = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

        int n = Integer.parseInt(firstMultipleInput[0]);
        int m = Integer.parseInt(firstMultipleInput[1]);

        List<String> board = IntStream.range(0, n).mapToObj(i -> {
          try {
```

```
                return bufferedReader.readLine();
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .collect(toList());

        int result = Result.queensBoard(board);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();
    } catch (IOException ex) {
        throw new RuntimeException(ex);
    }
});

bufferedReader.close();
bufferedWriter.close();
}
}
```

Super-Queens on a Chessboard

People on Mars have slightly different pieces on their Chessboard. One of them is a SuperQueen. A Super-Queen is a combination of a Queen and a Knight.

So, any of the following squares are in the "zone of power" of a Super-Queen.

1. A square in the same row or column as the Super-Queen
2. A square lying on a line drawn diagonally through the square on which the Super-Queen is. 3. A square lying in an 'L-Shape' with the Queen: This includes any square which is (2 rows, 1 column) away from the Queen or (1 row, 2 columns) away from the Queen.

So, if the Super-Queen is placed at the position 'q' marked on this chessboard below, the squares marked with hyphens '-' are squares threatened by possible 'attack' from the SuperQueen, and the squares marked by '0' are squares which are safe from the Super-Queen.

```
- O O - O O -
O - - - - - O
O - - - - - O
- - - q - - -
O - - - - - O
O - - - - - O
- O O - O O -
```

Task

Your tasks is to compute the number of ways to place N Super-Queens on an N x N Chessboard such that none of the Super-Queens are in conflict with each other. Ignore the fact that some of these arrangements are reflections and rotations of each other: all of them count as unique positionings.

Input Format

One Integer N (which is the number of rows in the chessboard).

Constraint

•

Output Format

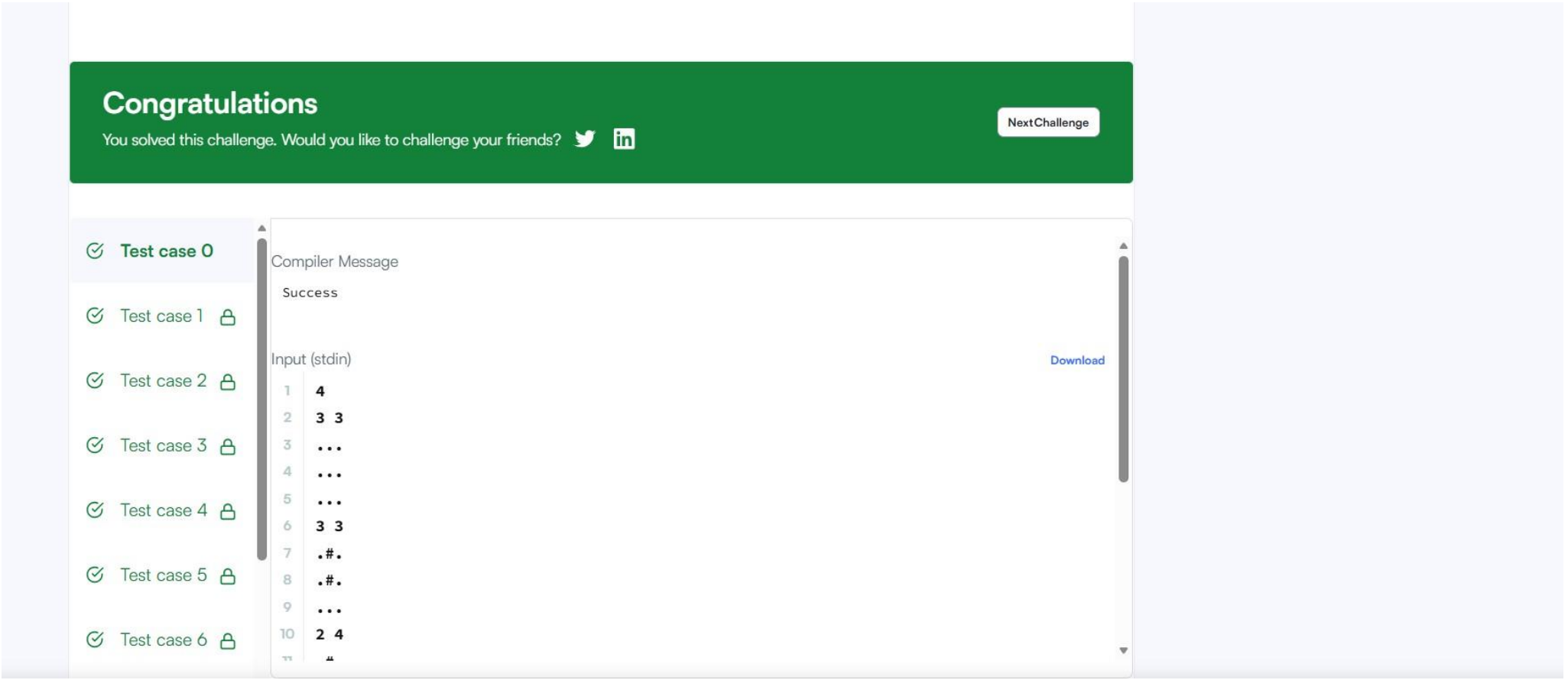
One Integer W, which is the number of ways to place N Super-Queens in the prescribed manner.

Sample Input

```
10
```

Sample Output

```
4
```



[Queens on Board | HackerRank]

DOMS | Page No.
Date / /

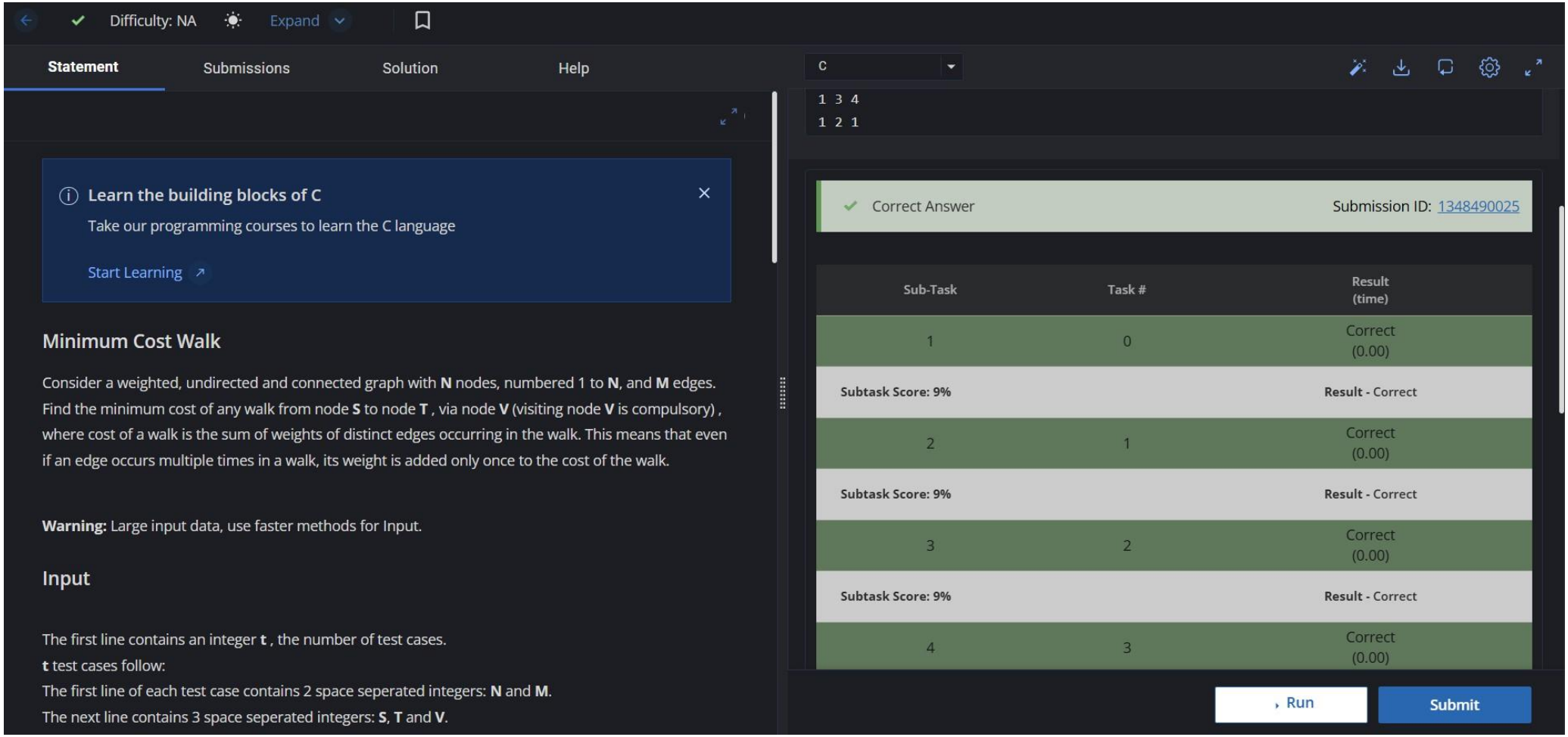
Task 2:- In what way is Solving N-Queens similar to how an AI agent solves Sudoku puzzles?

→ Solving N-Queens and Sudoku are fundamentally similar because both are classic Constraint Satisfaction Problem (CSP) that AI agent typically solve using a backtracking algorithm. In both puzzles, the AI place an element - either a queen on a chessboard or a number in a grid - and immediately checks if it violates specific rules. For N-Queens, the constraint is that no queens can share a row, column, or diagonal. For Soduku, numbers cannot repeat in a row, column, or Subgrid. If a placement breaks a rule, the AI 'backtracks' undoing the last move and exploring alternative placement until it discover a valid solution

Task 3 - Optimization (Informed Search)

Minimum Cost Walk

Consider a weighted, undirected and connected graph with **N** nodes, numbered 1 to **N**, and **M** edges. Find the minimum cost of any walk from node **S** to node **T** , via node **V** (visiting node **V** is compulsory) , where cost of a walk is the sum of weights of distinct edges occurring in the walk. This means that even if an edge occurs multiple times in a walk, its weight is added only once to the cost of the walk.



Minimum Cost Walk Practice Coding Problem

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Set sufficiently large constraints for typical CodeChef problems
#define MAX_N 500005
```

```
#define MAX_M 2000005
#define INF 10000000000000000LL // 1e18

int head[MAX_N];
int to[MAX_M];
long long weight[MAX_M];
int nxt[MAX_M];
int edge_cnt;

// Initialize graph adjacency list
void init_graph(int n) {
    for (int i = 0; i <= n; i++) {
        head[i] = -1;
    }
    edge_cnt = 0;
}

// Add an undirected edge
void add_edge(int u, int v, long long w) {
    to[edge_cnt] = v;
    weight[edge_cnt] = w;
    nxt[edge_cnt] = head[u];
    head[u] = edge_cnt++;

    to[edge_cnt] = u;
```

```
    weight[edge_cnt] = w;
    nxt[edge_cnt] = head[v];
    head[v] = edge_cnt++;
}

// --- MIN HEAP IMPLEMENTATION FOR DIJKSTRA ---
typedef struct {
    long long d;
    int u;
} HeapNode;

HeapNode heap[MAX_M];
int heap_sz;

void push(long long d, int u) {
    int i = ++heap_sz;
    while (i > 1 && heap[i / 2].d > d) {
        heap[i] = heap[i / 2];
        i /= 2;
    }
    heap[i].d = d;
    heap[i].u = u;
}

HeapNode pop() {
```

```
    HeapNode ret = heap[1];
    HeapNode temp = heap[heap_sz--];
    int i = 1, child = 2;
    while (child <= heap_sz) {
        if (child < heap_sz && heap[child + 1].d < heap[child].d) {
            child++;
        }
        if (temp.d <= heap[child].d) {
            break;
        }
        heap[i] = heap[child];
        i = child;
        child *= 2;
    }
    heap[i] = temp;
    return ret;
}
// ----------------------------------------

long long ds[MAX_N];
long long dt[MAX_N];
long long dv[MAX_N];

void dijkstra(int start, int n, long long* dist) {
    for (int i = 0; i < n; i++) {
```

```
        dist[i] = INF;
    }
    heap_sz = 0; // Clear the heap

    dist[start] = 0;
    push(0, start);

    while (heap_sz > 0) {
        HeapNode top = pop();
        long long d = top.d;
        int u = top.u;

        // Skip if we found a shorter path already
        if (d > dist[u]) continue;

        for (int e = head[u]; e != -1; e = nxt[e]) {
            int v = to[e];
            long long w = weight[e];
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                push(dist[v], v);
            }
        }
    }
}
```

```
void solve() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) return;

    int s, target, v;
    scanf("%d %d %d", &s, &target, &v);

    // 0-based indexing
    s--;
    target--;
    v--;

    init_graph(n);

    for (int i = 0; i < m; i++) {
        int a, b;
        long long w;
        scanf("%d %d %lld", &a, &b, &w);
        a--;
        b--;
        add_edge(a, b, w);
    }

    // Three shortest-path calculations
```

```
    dijkstra(s, n, ds);
    dijkstra(target, n, dt);
    dijkstra(v, n, dv);

    // Find the best meeting point
    long long answer = INF;
    for (int i = 0; i < n; i++) {
        long long cost = ds[i] + dt[i] + dv[i];
        if (cost < answer) {
            answer = cost;
        }
    }

    printf("%lld\n", answer);
}

int main() {
    // Fast I/O is less critical in C due to scanf/printf speed, but good practice
    int t;
    if (scanf("%d", &t) == 1) {
        while (t--) {
            solve();
        }
    }
    return 0;
```

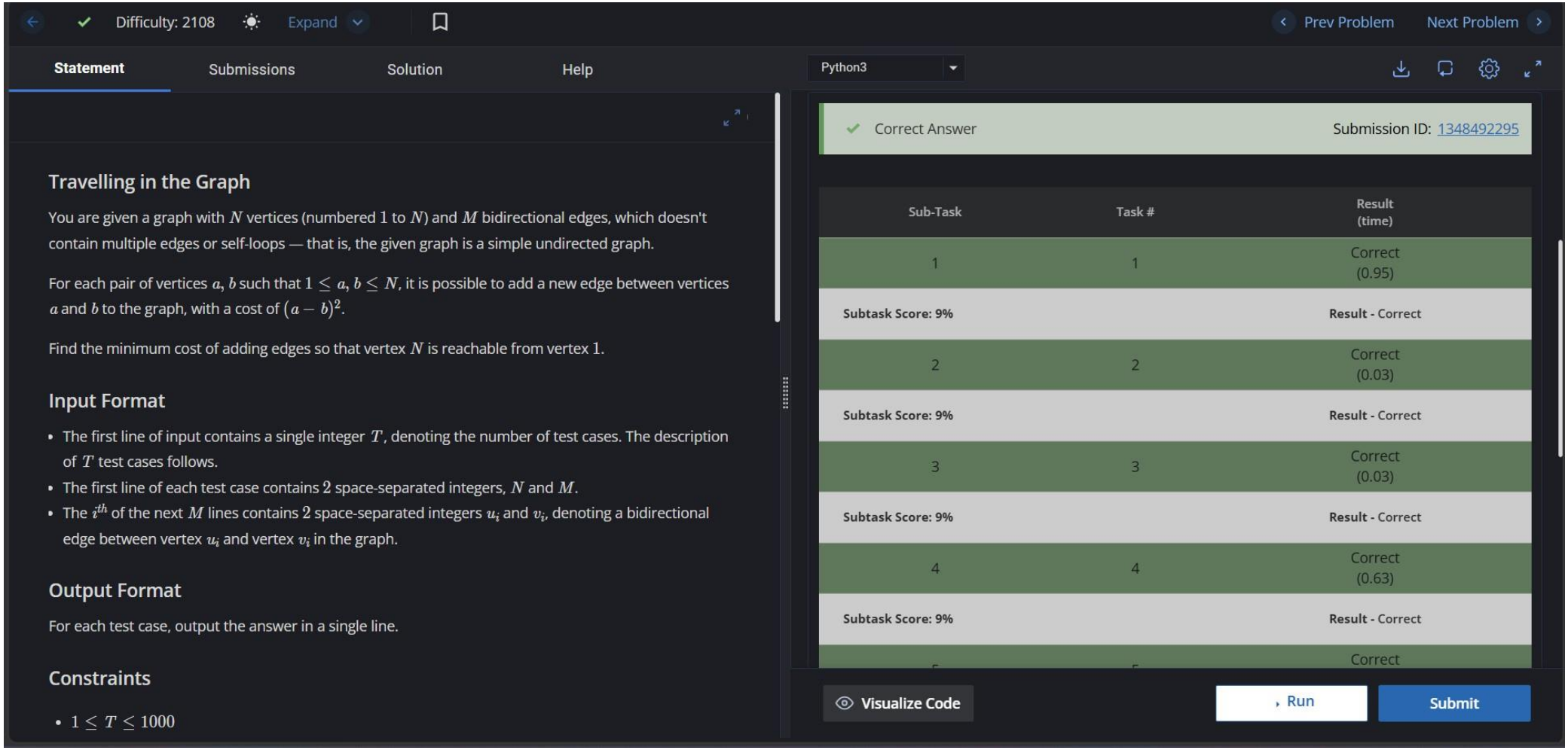
TASK 3:

Travelling in the Graph

You are given a graph with NN vertices (numbered 1 to NN) and MM bidirectional edges, which doesn't contain multiple edges or self-loops — that is, the given graph is a simple undirected graph.

For each pair of vertices a,ba,b such that 1≤a,b≤N1≤a,b≤N, it is possible to add a new edge between vertices aa and bb to the graph, with a cost of (a−b)2(a−b)2.

Find the minimum cost of adding edges so that vertex NN is reachable from vertex 1.



Travelling in the Graph Practice Coding Problem

```
import sys
from collections import deque


def solve():
    input = sys.stdin.buffer.readline
```

```
T = int(input())
answers = []

for _ in range(T):

    N, M = map(int, input().split())

    # Graph containing:
    # 1. Original edges with cost 0
    # 2. Edges between consecutive vertices with cost 1
    graph = [[] for _ in range(N)]

    for _ in range(M):
        u, v = map(int, input().split())

        u -= 1
        v -= 1

        # Original edge costs 0
        graph[u].append((v, 0))
        graph[v].append((u, 0))

    # We can add edge (i, i+1) with cost 1
    for i in range(N - 1):
```

```
    graph[i].append((i + 1, 1))
    graph[i + 1].append((i, 1))

# 0-1 BFS
dist = [10**18] * N
dist[0] = 0

dq = deque([0])

while dq:

    u = dq.popleft()

    if u == N - 1:
        break

    for v, cost in graph[u]:

        new_dist = dist[u] + cost

        if new_dist < dist[v]:

            dist[v] = new_dist

            if cost == 0:
```

```
                dq.appendleft(v)
            else:
                dq.append(v)

        answers.append(str(dist[N - 1]))

    print("\n".join(answers))


if __name__ == "__main__":
```

solve()

Task 3:- To do adapt an optimization algorithm for a drone, the navigation system must transition from a static 2D grid to a dynamic 3D environment. First, the algorithms State Space must expand to include a z-axis, allowing to calculate altitude adjustment alongside standard directional movement. When the vision or sensor system flag an obstruction, the algorithm instantly assigns an infinite "cost" to those specific 3D coordinate. This immediate data update forces the algorithm to recalculate, generating a safe, evasive flight path in real time -